Actividades prácticas: Colecciones y tablas

EJERCICIO 1. Implemente y verifique la clase *Historicos* que modela una colección de objetos de clase *Jugador*.

Historicos
<<Atributos de instancia>> jugadoresHistoricos: [] Jugador cant: entero
<<Constructor>> Historicos(n: entero) *<<Comandos>>* establecerJugador(j: Jugador) ordenar() *<<Consultas>>* estaLlena(): boolean cantPosiciones(): entero cantJugadores(): entero obtenerJugador(p: entero): Jugador mMayores(g, m: entero): boolean alMenosMMayores(g, m: entero): boolean goleadores(g: entero): Historico
Sólo las primeras *cant* posiciones del arreglo están ligadas.

Jugador
<<Atributos de instancia>> nombre: String nroCamiseta: entero posicion: entero golesConvertidos: entero partidosJugados: entero
<<Constructor>> Jugador(nom: String) *<<Comandos>>* establecerNroCamiseta(n: entero) establecerPosicion(n: entero) estableceGolesConvertidos(n: entero) establecerPartidosJugados(n: entero) aumentarGoles(n: entero) aumentarUnPartido() *<<Consultas>>* obtenerNombre(): String obtenerNroCamiseta(): entero obtenerPosicion(): entero obtenerGolesConverstidos(): entero obtenerPartidosJugados(): entero promedioGolesXPart(): entero masGoles(j: Jugador): boolean jugConMasGoles(j: Jugador): Jugador toString(): String clone(): Jugador equals(j: Jugador): boolean

- *Historicos(n: entero) .* Requiere n mayor a 0.
- *establecerJugador(j: Jugador)*. Requiere que la colección no esté llena y j ligada. Asigna j a la posición cant e incrementa cant en 1.
- *ordenar()*. Ordena la estructura por nombre de jugador usando el método de la burbuja.
- *obtenerJugador(p: entero): Jugador*. Requiere p válida.
- *mMayores(g, m: entero): boolean*. Retorna true si la colección mantiene exactamente m jugadores con más de g goles convertidos.
- *alMenosMMayores(g, m: entero): boolean*. Retorna true si la colección mantiene al menos m jugadores con más de g goles convertidos.
- *goleadores(g: entero): Historicos*. Retorna una nueva colección con los jugadores que hicieron más de g goles.

Existen muchos métodos de ordenamiento, investigue e implemente la estrategia del método Burbuja.

EJERCICIO 2. Implemente y verifique de acuerdo a la siguiente especificación:

MedicionServicios
<<Atributos de instancia>> gas: entero agua: entero electricidad: entero
<<Constructor>> MedicionServicios(g, a, e: entero) *<<Comandos>>* establecerGas(g: entero) establecerAgua(a: entero) establecerElectricidad(e: entero) *<<Consultas>>* obtenerGas(): entero obtenerAgual(): entero obtenerElectricidad(): entero costoServicios(): real

ConsumoEdificio
<<Atributos de instancia>> consumoXPiso: [] MedicionServicios
<<Constructor>> ConsumoEdificio(pisos: entero) *<<Comandos>>* registrarConsumo(m: MedicionServicios, p: entero): boolean eliminarConsumo(p: entero): boolean eliminarCeros() *<<Consultas>>* cantPosiciones(): entero cantRegistrosConsumo(): entero obtenerMedicionServicios(p: entero): MedicionServicios hayMayorConsumo(c: entero): boolean cuantasMayores(c: entero): entero primerPisoMayor(c: entero): entero ultimoPisoMayor(c: entero): entero nMayores(c: entero, n: entero): boolean todosMayores(c: entero): boolean todosMayores(m: MedicionServicios): boolean

En la clase *MedicionServicios*

- *costoServicios(): real*. Retorna gas*1.5 + agua*0.5 + electricidad*2.

La clase *ConsumoEdificio* modela una tabla de objetos de clase *MedicionServicios*.
Impleméntela considerando que desde las clases cliente, el primer piso se denota con 1.

- *ConsumoEdificio(pisos : entero).* Requiere pisos mayor a 1
- *registrarConsumo(m: MedicionServicios, p: entero): boolean .* Si p es un piso válido, asigna m al piso p y retorna true, sino retorna false.
- *eliminarConsumo(p: entero): boolean*. Si p es un piso válido, asigna una referencia nula al piso p y retorna true, sino retorna falso.
- *eliminarCeros()*. Asigna referencias nulas a todos los pisos asociados a Mediciones de Servicio que mantienen dos o tres valores de consumo en 0.
- *obtenerMedicionServicios(p: entero): MedicionServicios* . Requiere p válido
- *hayMayorConsumo(c: entero): boolean*. Retorna true si el costo de algún piso del edificio fue mayor a c.
- *cuantasMayores(c: entero): entero*. Calcula cuántos pisos del edificio tuvieron un costo mayor a c.
- *primerPisoMayor(c : entero): entero*. Retorna el primer piso del edificio que tiene un costo mayor a c. Si no hay ningún piso con costo mayor a c retorna -1.
- *ultimoPisoMayor(c : entero): entero*. Retorna el último piso del edificio que registra un costo mayor a c. Si no hay ningún piso con costo mayor a c retorna -1.
- *nMayores(c: entero, n: entero): boolean*. Determina si hay n pisos con costos mayores a c.
- *todosMayores(c : entero): boolean*. Retorna true si todos los pisos tuvieron un costo mayor a c.

- *todosMayores(m: MedicionServicios): boolean*. Determina si en cada uno de los pisos en los que se registró una medición, los consumos de gas, agua y electricidad son mayores a los registrados en la medición m. Requiere m ligada.

EJERCICIO 3. Implemente y verifique la clase *PlayaSurtidores* que modela a una tabla de surtidores.

Surtidor
<<Atributos de clase>> maximaCarga: entero *<<atributos de instancia>>* cantGasoil: entero cantSuper: entero cantPremium: entero
<<Constructor>> Surtidor() *<<Comandos>>* llenarDepositoGasoil() llenarDepositoSuper() llenarDepositoPremium() extraerGasoil(litros: entero) extraerSuper(litros: entero) extraerPremium(litros: entero) *<<Consultas>>* obtenerMaximaCarga(): entero obtenerLitrosGasoil(): entero obtenerLitrosSuper(): entero obtenerLitrosPremium(): entero

PlayaSurtidores
<<Atributos de instancia>> Tabla: [] Surtidor
<<Constructores>> PlayaSurtidores(n: entero) PlayaSurtidores() *<<Comandos>>* establecerSurtidor(s: Surtidor, p: entero) establecerSurtidor(s: Surtidor) llenaDepositos() agrupaSurtidores() *<<Consultas>>* cantPosiciones(): entero cantSurtidores(): entero obtenerSurtidor(p: entero): Surtidor cantMaximaCarga(): entero mismosNulos(ps: PlayaSurtidores): boolean clone(): PlayaSurtidores nLigados(n:entero):PlayaSurtidores

- *PlayaSurtidores()*. Crea una tabla con capacidad para 10 surtidores. Todas las posiciones deben ligarse a surtidores con máxima carga en cada tipo de combustible.
- *PlayaSurtidores(n:entero)*. Requiere n>0. Crea una tabla con capacidad para n surtidores. La tabla se crea con todas las posiciones nulas.
- *establecerSurtidor(s: Surtidor, p: entero)*. Requiere p válida. Asigna s a la posición p.
- *establecerSurtidor(s: Surtidor)*. Requiere que la tabla no esté llena. Asigna s a la primera posición libre.
- *llenaDepositos()*. Llena los depósitos de cada uno de los surtidores que haya en la tabla.
- *agruparSurtidores()*. Arrastra las referencias no ligadas a las últimas posiciones de modo que todas las referencias ligadas ocupen posiciones consecutivas al principio de la estructura.
- *obtenerSurtidor(p: entero): Surtidor*. Requiere p válida.
- *cantMaximaCarga(): entero*. Retorna la cantidad de surtidores que tienen la máxima carga en cada uno de los tres depósitos.
- *mismosNulos(ps: PlayaSurtidores): boolean*. Retorna true si las dos tablas mantienen referencias nulas en exactamente las mismas posiciones.
- *clone(): PlayaSurtidores*. Implementa clone superficial. Genera una nueva tabla con el mismo tamaño y las mismas referencias que la playa de surtidores que recibe el mensaje.

- *nLigados(n:entero): PlayaSurtidores*. Si la playa de surtidores tiene una secuencia de al menos n posiciones ligadas, genera una nueva playa con dicha secuencia. Si la secuencia no existe retorna nulo, si hay más de una secuencia genera una nueva playa con la primera secuencia.

Observe que la funcionalidad de los comandos establecer en la clase *PlayaSurtidores* permite tanto ocupar una posición con un surtidor, como liberar la posición si el parámetro s no está ligado.

EJERCICIO 4. Implemente y verifique las clases *Territorio* y *ColeccionRefugios* de acuerdo a la siguiente especificación.

Territorio
<<*Atributos de instancia*>> tablero: [] [] Refugio
<<*Constructor*>> Territorio(nf, nc: entero) <<*Comandos*>> establecerRefugio(r: Refugio, f, c: entero): boolean <<*Consultas*>> cantFilas():entero cantColumnas():entero cantRefugios():entero filaMasPoblada(): entero maximoNRefugiosHabitables(f,n:entero):boolean cantConDisponibilidadMayor(m:entero):entero territorioEspejo():boolean coleccionFila():ColeccionRefugios clone():Territorio

ColeccionRefugios
<<*Atributo de instancia*>> col: [] Refugio cant: entero
<<*Constructor*>> ColeccionRefugios(n: entero) <<*Comandos*>> insertarRefugio(r: Refugio) eliminarEscasaSupervivencia() <<*Consultas*>> tamanoColeccion():entero cantRefugios(): entero estaLlena(): boolean obtenerRefugio(pos: entero): Refugio hayNSeguidosHabitables(n: entero): boolean equals(c: ColeccionRefugios): boolean
Las primeras cant posiciones del arreglo están ligadas, las demás no lo están.

Refugio
<<*Atributos de clase*>> capacidadAlacena = 20 cantidadCamas = 10
<<*Atributos de instancia*>> alimentos, bebidas, camasDisponibles: entero
<<*Constructor*>> Refugio(a, b, c: entero) <<*Comandos*>> consumirAlimento() consumirBebida() ocuparCama(): boolean desocuparCama(): boolean reponerAlimentos(n: entero): boolean reponerBebidas(n: entero): boolean <<*Consultas*>> obtenerAlimentos(): entero obtenerBebidas(): entero obtenerCamas(): entero obtenerCapacidadAlacena(): entero esHabitable(): boolean disponibilidad(): entero diasSupervivencia(): entero mayorAlimentos(r: Refugio): boolean equals(c: Refugio): boolean clone(): Refugio toString(): String

En la clase *Territorio* el constructor requiere nf y nc mayores a 0.

- *establecerRefugio(r: Refugio, f, c: entero): boolean*. Asigna el refugio r a la posición f, c y retorna true si 0 <= f < cantFilas() y 0 <= c < cantColumnas(), sino retorna false.
- *cantFilas(): entero*. Retorna la cantidad de filas del arreglo.
- *cantColumnas(): entero*. Retorna la cantidad de columnas.
- *cantRefugios(): entero*. Retorna la cantidad de componentes ligadas del arreglo.
- *filaMasPoblada(): entero*. Retorna el número de la fila con mayor número de refugios; si hay más de una con ese valor retorna la primera.
- *maximoNRefugiosHabitables(f, n: entero): boolean*. Retorna true si la fila f contiene, a lo sumo n refugios habitables.
- *cantConDisponibilidadMayor(m: entero): entero*. Computa cuántas columnas tienen un refugio con disponibilidad mayor a m.
- *territorioEspejo():boolean*. Retorna true si la cantidad de columnas es impar y el territorio a la derecha de la columna central es la imagen espejo del territorio a la izquierda de la columna central, comparando el estado interno de los refugios.
- *coleccionFila(): ColeccionRefugios*. Crea y retorna una colección con la primera referencia ligada de cada fila del territorio que recibe el mensaje. Si una fila del territorio no tiene ninguna referencia ligada, no se inserta ningún refugio correspondiente a esa fila.
- *clone(): Territorio*. Implementa clone en profundidad.

En la clase *ColeccionRefugios*, el constructor requiere n>0.

- *insertarRefugio(r: Refugio)*. Si la colección no está llena, asigna el refugio r a la primera posición libre en la colección e incrementa la cantidad de posiciones ligadas. Requiere r ligado.
- *eliminarEscasaSupervivencia()*. Elimina de la colección todos los refugios que tienen a lo sumo 1 día de supervivencia y arrastra de modo que el arreglo siga modelando a una colección.
- *obtenerRefugio(pos: entero): Refugio* . Si 0 <= pos < cantRefugios() Retorna el refugio de la posición pos, sino retorna nulo.
- *hayNSeguidosHabitables(n: entero): boolean*. Retorna true si la colección mantiene al menos n refugios habitables en posiciones consecutivas.
- *equals(c: ColeccionRefugios): boolean*. Se implementa en profundidad, es decir, retorna true si las dos colecciones tienen el mismo tamaño, la misma cantidad de refugios y los refugios que ocupan la misma posición en los dos territorios, son equivalentes.

EJERCICIO 5. Implemente y verifique la clase *PolizasOrdenadas* que modela una <u>colección</u> ordenada, de acuerdo a la siguiente especificación

PolizasOrdenadas
<<*Atributos de instancia*>> polOrd: Poliza [] n: entero
<<*Constructor*>> PolizasOrdenadas(max: entero) <<*Comandos*>> insertar(pol: Poliza) actualizar(d, h, p: entero) copy(p: PolizasOrdenadas) <<*Consultas*>> cantPolizas(): entero cantElementos(): entero estaLlena(): boolean existePoliza(nro: entero): boolean existePoliza(pol: Poliza): boolean obtenerPoliza(pos:entero):Poliza intercalar(p: PolizasOrdenadas): PolizasOrdenadas equals(p: PolizasOrdenadas): boolean
Las primeras n posiciones del arreglo están ligadas y las que siguen no están ligadas. Los elementos en todo momento están ordenados por número de póliza.

Poliza
<<*Atributos de instancia*>> nroPoliza: entero incendio: real robo: real activa: boolean
<<*Constructor*>> Poliza(np: entero) Poliza(np: entero, i: real, r: real) <<*Comandos*>> establecerIncendio(m: real) establecerRobo(m: real) actualizarPorcentaje(p: entero) activar() desactivar() <<*Consultas*>> obtenerNroPoliza(): entero obtenerIncendio(): real obtenerRobo(): real obtenerCostoPoliza(): real estaActiva(): boolean equals(p: Poliza): boolean
Todos los servicios requieren que los parámetros numéricos mantengan valores positivos.

- *PolizasOrdenadas(max: entero).* Requiere max >0.
- *insertar(pol: Poliza)*. Requiere pol ligado, que la colección no tenga una póliza con el mismo número que pol y no está completa. Agrega la nueva póliza de manera tal que los elementos quedan ordenados por número de póliza.
- *actualizar(d, h, p: entero)*. Actualiza todas las pólizas con costo mayor a d y menor a h en un p por ciento. Requiere d<=h.
- *copy(p: PolizasOrdenadas)*. Implementa copia superficial. Si ambas colecciones tienen la misma cantidad de elementos, copia cada póliza de p en la colección que recibe el mensaje. Si la cantidad de elementos no coincide no provoca ningún cambio.
- *existePoliza(nro: entero): boolean*. Retorna true si la colección contiene una póliza con número igual a nro.
- *existePoliza (pol:Poliza):boolean*. Retorna true si la colección contiene una póliza con el mismo estado interno que pol. Implementa búsqueda binaria iterativa.
- *obtenerPoliza(pos:entero):Poliza*. Retorna la póliza asignada a la posición pos si pos es válida, si no retorna nulo.
- *intercalar(p: PolizasOrdenadas): PolizasOrdenadas*. Retorna una nueva colección que se genera intercalando ordenadamente las dos colecciones, recorriendo solo una vez cada una de ellas. Si hay

dos pólizas con el mismo número agrega a la nueva colección la póliza de la colección que recibió el mensaje.

- *equals(p: PolizasOrdenadas): boolean*. Si la comparación es posible implementa igualdad superficial en caso contrario retorna false.

Observación: El método intercalar agrega cada nuevo elemento en la nueva colección siempre al final: no necesita recorrerla para buscar dónde insertar.

EJERCICIO 6. La clase *ColeccionPoligonos* modelada en el diagrama permite almacenar la representación de una colección de polígonos. Cada vez que se inserta un polígono en la colección se incrementa el valor de *n*. Si se almacenan *n* polígonos, se ocupan las primeras *n* componentes del arreglo. Cuando se elimina un polígono de la colección, los elementos se **"arrastran"** para seguir estando comprimidos. Cuando un servicio elimina varios, se arrastra una sola vez. Desde la clase cliente de la Colección no se acceden a los polígonos por posición. El servicio *eliminarPoligonos* elimina de la colección de polígonos todos los polígonos que tienen *n* lados, dejando el arreglo comprimido.

Cada polígono de *n* lados se representa como una colección de *n* puntos. El servicio *obtenerPunto* retorna el punto que ocupa la posición *p* en la secuencia, **comenzando desde la posición 1** y asumiendo *p* válida. Cada lado del polígono queda determinado por los elementos que ocupan las posiciones *i* e *i+1* en la secuencia, excepto uno de los lados que se forma con los puntos que ocupan la posición 1 y *n*. Cada vez que se inserta un punto en un polígono, se incrementa el valor de *n.* Si se almacenan *n* puntos, se ocupan las primeras *n* componentes del arreglo. Cuando se elimina un punto del polígono, los elementos se "**arrastran**" para seguir estando comprimidos y en la misma secuencia que fueron ingresados. Dos polígonos son iguales si mantienen dos secuencias de puntos equivalentes.

Poligono
<<Atributos de instancia>> pol [] Punto n: entero
<<Constructor>> Poligono(max: entero) *<<Comandos>>* insertar(p: Punto) eliminar(p: Punto) *<<Consultas>>* cantPuntos(): entero estaLlena(): boolean obtenerPunto(p: entero): Punto existePunto(p: Punto): boolean perimetro(): real equals(p:Poligono):boolean
Sólo las primeras *n* posiciones del arreglo *pol* están ligadas. El orden de los puntos determina los vértices del polígono.

ColeccionPoligonos
<<Atributos de instancia>> cp: [] Poligono n: entero
<<Constructor>> ColeccionPoligonos(max: entero) *<<Comandos>>* insertar(p: Poligono) eliminar(p: Polígono) eliminarPoligonosNLados(n: entero) *<<Consultas>>* cantPoligonos(): entero estaLlena(): boolean existePoligono(pol: Poligono): boolean contarPoligonosPunto(p: Punto): entero mayorPerimetro(): real
Sólo las primeras *n* posiciones del arreglo están ligadas.

a) Implemente las clases *Poligono* y *ColecciónPoligonos* teniendo en cuenta:

En la clase *Poligono*:

- *Poligono(max: entero)*. Requiere max >2.
- *obtenerPunto(p:entero): Punto*. Requiere que p sea una posición valida.

En la clase *ColecciónPoligonos*:

- *ColeccionPoligonos(max: entero)*. Requiere max >0.
- *mayorPerimetro(): real*. Retorna el mayor perímetro de la colección de polígonos, o 0 si la colección está vacía.
- *contarPoligonosPunto(p: Punto): entero*. Cuenta los polígonos que incluyen al punto p

b) Implemente una clase *Tester* con casos de prueba significativos y mensajes adecuados.